

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 15 | Heaps, Priority Queues and Heap Sort

Week 15 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 15 at a glance

Item	Details
Lab title	Heaps, Priority Queues and Heap Sort
Week	15 of 16
Prerequisites	Lab 11 (Queues), Lab 13 (Binary Trees, array representation)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab15_Instructor.ipynb , Lab15_Student.ipynb
Next lab	Lab 16: Graphs, BFS, DFS and Shortest Paths

1. Learning Objectives

By the end of this lab, you will be able to:

1. Explain the **heap property** for **max-heaps** and **min-heaps** and why a heap is a **complete binary tree**.
2. Store a heap in an **array** and navigate it with index formulas.
3. Implement **insert (sift-up)**, **extract (sift-down)** and **peek** in $O(\log n)$ and $O(1)$.
4. Build a heap from an array in **$O(n)$** with bottom-up heapify.
5. Implement **Heap Sort** in place and analyse it.
6. Use Python's **heapq** module, including max-heap tricks and **nlargest**.
7. Solve top-k, k-way merge and running-median problems with heaps.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement heaps and heap-based algorithms	Apply (C3)
CLO-3	Analyse heap operations and choose heaps for priority problems	Analyse (C4)

2. Motivation: Always Serve the Most Important First

In Lab 11 you met the **priority queue**: the element with the highest priority leaves first. How do we build one efficiently? A sorted list makes insertion $O(n)$; an unsorted list makes removal $O(n)$. A **heap** does **both in $O(\log n)$** and shows the top priority in **$O(1)$** .

Implementation	insert	get max/min	remove max/min
Unsorted list	$O(1)$	$O(n)$	$O(n)$
Sorted list	$O(n)$	$O(1)$	$O(1)$
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$
Binary heap	$O(\log n)$	$O(1)$	$O(\log n)$

2.1 Heaps in Daily Life

Situation	Priority
Hospital emergency (triage)	Most serious patient first

Situation	Priority
Airport boarding	Business class and special assistance first
Your to-do list before exams	Nearest deadline first
Ride-hailing (Careem / inDrive)	Nearest available driver first
University merit list (top 50 admissions)	Highest aggregate first

2.2 Heaps in Industry

- **Operating systems** schedule processes and timers with priority queues.
- **Shortest-path algorithms** (Dijkstra, A*) in Google Maps and games pick the closest unexplored location from a heap (Lab 16).
- **Top-K queries:** trending hashtags, top-selling products, most active users computed over huge streams with small heaps.
- **Event-driven simulation** and **job schedulers** (cron systems, Kubernetes) process events in time order with heaps.
- **Data compression:** Huffman coding (ZIP, JPEG, MP3) repeatedly combines the two least frequent symbols using a min-heap.

2.3 Heaps in Research

- **Search algorithms in AI:** best-first search, A* and beam search (used when language models generate text) keep candidates in priority queues.
- **Nearest-neighbour search** for vector databases keeps the current best k results in a heap.
- **Streaming algorithms** compute medians and percentiles on unbounded data with heaps.

2.4 Why It Matters in 2026–2030

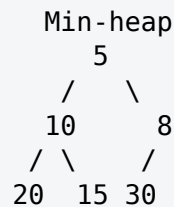
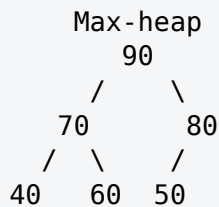
Prioritisation at scale

As systems handle millions of requests and AI models generate many candidate answers, deciding **what to process first** becomes critical. Heaps are the simple, fast answer to “give me the best k items” and “what is next?”. They also give you an $O(n \log n)$ sorting algorithm that needs no extra memory.

3. Part A: What Is a Heap?

A **binary heap** is a binary tree with two properties:

1. **Shape property:** it is a **complete binary tree** (all levels full except possibly the last, which is filled from the left).
2. **Heap property:** in a **max-heap**, every parent $\geq$ its children, so the **largest** key is at the root. In a **min-heap**, every parent $\leq$ its children, so the **smallest** key is at the root.



array: [90, 70, 80, 40, 60, 50]

array: [5, 10, 8, 20, 15, 30]

! A heap is not sorted

Only parent–child order is guaranteed. Siblings and cousins can be in any order. 80 is in level 1 and 60 in level 2, but nothing says $70 > 80$. This partial order is what makes heaps fast to maintain.

3.1 Array Representation

Because a heap is complete, we store it in an array **without gaps** (as in Lab 13). For index i : **left** = $2i + 1$, **right** = $2i + 2$, **parent** = $(i - 1) // 2$. The height of a heap with n nodes is $\lfloor \log_2 n \rfloor$, which is why its operations are $O(\log n)$.

⚙️ 4. Part B: Heap Operations (Min-Heap)

4.1 Insert: Add at the End, then Sift Up

Put the new key at the end of the array (keeps the shape). Then, while it is smaller than its parent, **swap it with the parent** (“bubble up”).

Sift-up

Insert 3 into [5, 10, 8, 20, 15, 30]

[5, 10, 8, 20, 15, 30, 3] 3 at index 6, parent index 2 (8) -> swap

[5, 10, 3, 20, 15, 30, 8] 3 at index 2, parent index 0 (5) -> swap

[3, 10, 5, 20, 15, 30, 8] 3 is the new root

4.2 Extract Min: Move the Last to the Root, then Sift Down

The minimum is at index 0. Replace the root with the **last element**, remove the last slot, and then, while the new root is larger than its **smaller child**, swap them (“bubble down”).

```
class MinHeap:
    def __init__(self):
        self.a = []

    def __len__(self):
        return len(self.a)

    def peek(self):
        return self.a[0] if self.a else None           # 0(1)

    def push(self, x):
        self.a.append(x)
        self._sift_up(len(self.a) - 1)                # 0(log n)

    def pop(self):
        if not self.a:
            raise IndexError("pop from empty heap")
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._sift_down(0)
        return top

    def _sift_up(self, i):
        while i > 0:
            parent = (i - 1) // 2
            if self.a[i] < self.a[parent]:
                self.a[i], self.a[parent] = self.a[parent], self.a[i]
                i = parent
            else:
                break

    def _sift_down(self, i):
        n = len(self.a)
        while True:
            smallest, l, r = i, 2 * i + 1, 2 * i + 2
            if l < n and self.a[l] < self.a[smallest]:
                smallest = l
            if r < n and self.a[r] < self.a[smallest]:
                smallest = r
            if smallest == i:
                break
            self.a[i], self.a[smallest] = self.a[smallest], self.a[i]
            i = smallest

h = MinHeap()
```

```
for x in [20, 5, 15, 30, 8, 10]:  
    h.push(x)  
    print(f"push {x:>2}: {h.a}")  
print("pop order:", [h.pop() for _ in range(len(h))])
```

Operation	Time	Why
peek	$O(1)$	Root of the heap
push	$O(\log n)$	Sift-up along one path
pop	$O(\log n)$	Sift-down along one path
build (heapify)	$O(n)$	Most nodes are near the bottom
search for arbitrary key	$O(n)$	Heaps are not ordered for searching

5. Part C: Building a Heap in $O(n)$

Inserting n items one by one costs $O(n \log n)$. **Bottom-up heapify** is faster: leaves are already heaps, so call sift-down on every non-leaf node from index $n/2 - 1$ down to 0. Most nodes are near the bottom and sift down only a little, giving a total of $O(n)$.

```
def heapify(arr):                                # turn arr into a MIN-heap in place
    h = MinHeap()
    h.a = arr
    for i in range(len(arr) // 2 - 1, -1, -1):
        h._sift_down(i)
    return arr

data = [9, 4, 7, 1, 8, 2, 6, 3]
print(heapify(data))
```

6. Part D: Heap Sort

Heap sort uses a **max-heap**: build it in $O(n)$, then repeatedly **swap the root (maximum) with the last element of the heap**, shrink the heap by one, and sift down. The array becomes sorted in ascending order **in place**.

```
def sift_down_max(a, i, n):
    while True:
        largest, l, r = i, 2 * i + 1, 2 * i + 2
        if l < n and a[l] > a[largest]:
            largest = l
        if r < n and a[r] > a[largest]:
            largest = r
        if largest == i:
            return
        a[i], a[largest] = a[largest], a[i]
        i = largest

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):          # 1. build a max-heap:  $O(n)$ 
        sift_down_max(a, i, n)
    for end in range(n - 1, 0, -1):              # 2. move max to the end:  $n \times O(\log n)$ 
        a[0], a[end] = a[end], a[0]
        sift_down_max(a, 0, end)
    return a

print(heap_sort([12, 3, 17, 8, 34, 25, 1]))
```

Property	Heap sort
Time (all cases)	$O(n \log n)$
Extra space	$O(1)$
Stable?	No
Used in	Introsort's fallback (C++ <code>std::sort</code>), embedded systems



7. Part E: Python's `heapq` Module

`heapq` provides a **min-heap** on a plain list. For a **max-heap**, push negated values or tuples with negated priority.

```
import heapq

marks = [67, 89, 45, 92, 78, 55, 81]
heap = marks[:]
heapq.heapify(heap)                # O(n)
heapq.heappush(heap, 50)
print("smallest:", heap[0], "| popped:", heapq.heappop(heap))
print("top 3:", heapq.nlargest(3, marks), "| bottom 2:",
      heapq.nsmallest(2, marks))

max_heap = []
for m in marks:
    heapq.heappush(max_heap, -m)    # negate for max-heap
print("max:", -heapq.heappop(max_heap))

tasks = []
heapq.heappush(tasks, (2, "Prepare viva"))
heapq.heappush(tasks, (1, "Submit DSA lab"))
heapq.heappush(tasks, (3, "Watch PSL match"))
while tasks:
    print(heapq.heappop(tasks))
```



8. Part F: Classic Heap Patterns

8.1 Top-K from a Stream: Keep a Min-Heap of Size K

To keep the k largest values seen so far, keep a **min-heap of size k** . Its root is the smallest of the top k ; a new value enters only if it beats that root. Time $O(n \log k)$, memory $O(k)$: perfect for huge streams.

```
def top_k(stream, k):
    heap = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)      # pop smallest, push x
    return sorted(heap, reverse=True)

import random
scores = (random.randint(0, 10_000) for _ in range(100_000))  # a generator:
a "stream"
print(top_k(scores, 5))
```

8.2 Running Median with Two Heaps

Keep the smaller half of the numbers in a **max-heap** and the larger half in a **min-heap**, balanced in size. The median is at the top of one or both heaps. Each new number costs $O(\log n)$.

```
class RunningMedian:
    def __init__(self):
        self.low = []          # max-heap (store negatives)
        self.high = []         # min-heap

    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))  # move the
largest of low
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def median(self):
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2

rm = RunningMedian()
for bpm in [72, 90, 65, 80, 100, 70]:
    rm.add(bpm)
    print(f"added {bpm:>3} -> median {rm.median()}")
```

9. C++ Corner

C++

```
#include <iostream>
#include <queue>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    priority_queue<int> maxHeap; // MAX-heap
    // by default
    priority_queue<int, vector<int>, greater<int>> minHeap; // MIN-heap
    for (int x : {20, 5, 15, 30, 8}) { maxHeap.push(x); minHeap.push(x); }
    cout << maxHeap.top() << " " << minHeap.top() << endl; // 30 5

    vector<int> v = {12, 3, 17, 8, 34, 25, 1};
    make_heap(v.begin(), v.end()); // O(n) build, max-heap
    sort_heap(v.begin(), v.end()); // heap sort
    for (int x : v) cout << x << " ";
    cout << endl;

    // Patients: (severity, name); highest severity first
    priority_queue<pair<int, string>> er;
    er.push({2, "Ali"}); er.push({5, "Sara"}); er.push({3, "Usman"});
    while (!er.empty()) { cout << er.top().second << " "; er.pop(); }
}
```

Python vs C++ defaults

Python **heapq** is a **min-heap**; C++ **priority_queue** is a **max-heap**. Mixing them up is a very common bug when translating code.

10. Common Mistakes

Mistake	Effect	Fix
Thinking a heap is sorted	Wrong answers from heap[1], heap[2]...	Only heap[0] is guaranteed
Forgetting to sift after changing an element	Heap property broken	Sift up or down after every change
Sift-down comparing with only the left child	Wrong child swapped	Pick the smaller (or larger) of both children
Using a max-heap for top-K largest	$O(n \log n)$ and $O(n)$ memory	Use a min-heap of size k

Mistake	Effect	Fix
Pushing tuples with uncomparable items	TypeError when priorities tie	Add a counter: (priority, count, item)

11. Lab Tasks

Solve all tasks in `Lab15_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Is It a Heap?

Write `is_min_heap(arr)` that checks whether an array satisfies the min-heap property (every parent $\leq$ its children). It must be $O(n)$.

```
[1, 3, 2, 7, 4] → True, [1, 3, 2, 0] → False
```

Task 2 Last Stone Weight

You have stones with given weights. Each turn, take the **two heaviest** stones and smash them: if equal, both disappear; otherwise the lighter is destroyed and the heavier loses that weight. Return the weight of the last stone (0 if none). Use `heapq` as a max-heap.

```
[2, 7, 4, 1, 8, 1] → 1
```

Task 3 Max-Heap Class with Update

Write `MaxHeap` (your own, not `heapq`) with `push`, `pop`, `peek` and `__len__`. Then add `increase_key(i, new_value)`, which raises the value at index `i` and restores the heap (sift up).

Priority updates are needed in schedulers and in Dijkstra's algorithm.

Task 4 Top-K Frequent Hashtags

Given a list of hashtags, return the `k` most frequent ones (ties broken alphabetically). Count with a dictionary, then use a heap (`heapq.nsmallest` with key `(-count, tag)` or your own size-`k` heap). Do not sort the whole dictionary.

```
["#PSL", "#DSA", "#PSL", "#0kara", "#DSA", "#PSL"], k = 2 → ["#PSL", "#DSA"]
```

Task 5 Merge K Sorted Result Lists

Each of `k` sections has a merit list sorted in ascending order. Merge them into one sorted list in $O(N \log k)$ (N = total items) using a min-heap holding the current front of each list as `(value, list_index, element_index)`.

This is exactly how databases merge sorted runs during external sorting.

```
[[1, 4, 7], [2, 5, 8], [0, 9]] → [0, 1, 2, 4, 5, 7, 8, 9]
```

Task 6 ● K Closest Petrol Pumps

Petrol pumps are at coordinates (x, y) relative to your car at $(0, 0)$. Return the k closest pumps (by Euclidean distance; ties by coordinates), using a **max-heap of size k** so that the solution is $O(n \log k)$.

Hint: compare squared distances (no square roots needed); push $(-dist^2, -x, -y)$ into `heapq`.

```
[(1, 3), (-2, 2), (5, 8), (0, 1)], k = 2 → [(0, 1), (-2, 2)]
```

Task 7 ● CPU Task Scheduler Simulation

Tasks arrive as $(arrival_time, duration, name)$. A single CPU always runs, among the tasks that have **already arrived**, the one with the **shortest duration** (ties: earlier arrival, then name). Tasks run to completion. Return the order of task names.

Sort tasks by arrival; keep a min-heap of available tasks keyed by $(duration, arrival, name)$; when the CPU is idle and nothing is available, jump time to the next arrival.

```
[(0, 8, "A"), (1, 4, "B"), (2, 9, "C"), (3, 5, "D")] → ["A", "B", "D", "C"]
```



12. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub `DSA-Labs/Lab15` folder.

#	Problem	Level	Concept practised
1046	Last Stone Weight	Easy	Max-heap (Task 2)
703	Kth Largest Element in a Stream	Easy	Min-heap of size k
215	Kth Largest Element in an Array	Medium	Heap vs quickselect
347	Top K Frequent Elements	Medium	Counting + heap (Task 4)
973	K Closest Points to Origin	Medium	Max-heap of size k (Task 6)
23	Merge k Sorted Lists	Hard	K-way merge (Task 5)

#	Problem	Level	Concept practised
295	Find Median from Data Stream	Hard	Two heaps
1834	Single-Threaded CPU	Medium	Scheduling (Task 7)



13. Home Task

1. Finish all tasks and push `Lab15_Student.ipynb` to GitHub.
2. Prove (or explain with a table) why bottom-up heapify is $O(n)$: count how many nodes are at each height and how far each can sift down.
3. Implement a **d-ary heap** (each node has d children) and compare the number of comparisons for $d = 2, 3, 4$ on 10,000 pushes and pops.
4. **Mini-project: Hospital Triage System.** Patients arrive with severity 1–10 and arrival time. Use a heap so the most severe is treated first and equal severities are treated in arrival order. Support “condition worsened” (increase severity) by marking old entries invalid (lazy deletion). Report average waiting time per severity.
5. Solve at least 3 LeetCode problems from the list.

? 14. Viva Questions

1. What are the two properties of a binary heap?
2. What is the difference between a max-heap and a min-heap?
3. Why can a heap be stored in an array without gaps?
4. Give the index formulas for parent and children.
5. Explain sift-up during insertion.
6. Explain extract-max/min and sift-down.
7. Why is building a heap bottom-up $O(n)$ instead of $O(n \log n)$?
8. Explain heap sort. Is it stable? Is it in-place?
9. How do you make a max-heap with Python’s `heapq`?
10. How would you find the top 10 scores in a stream of one billion scores?



15. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



16. References and Further Learning

- Python `heapq` documentation: docs.python.org/3/library/heapq.html
- Heap animations: visualgo.net/en/heap
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.